

Trabalho Extra - Criptografia

Mateus Lima Rodrigues, 536334

Colocando aqui o link do [Repositório do Trabalho](#), caso não seja possível enviar os arquivos CSV gerados pelo sigaa, adicionei todos na pasta específica de cada desafio

Desafio A: Quebrando a Resistência a Colisões:

A propriedade de resistência a colisões diz que deve ser computacionalmente inviável encontrar dois inputs distintos, nesse caso duas strings x e y tal que o hash dessas duas strings seja o mesmo, para esse exemplo de apenas 4 bytes de digest conseguimos encontrar facilmente essa quebra na resistência a colisões

Para demonstrar isso, fiz um código python que gera strings aleatórias, calcula o seu hash usando a biblioteca padrão [Hashlib](#), verifica se o hash é novo e salva ele no set, se o hash nunca tiver sido lido, ele salva o hash em um arquivo csv para garantir a persistência dos dados (fazer isso gerou um código bem mais demorado que o normal, mas foi bom para ver diretamente o que está acontecendo) e finalmente, se o hash já existir ele faz uma verificação mais aprofundada, primeiro ele compara a string, pois mesmo se forem os mesmos hashes, sendo strings iguais a colisão não é aceita então ele volta a busca, mas se as strings forem diferentes, achamos a nossa colisão

Ao executar, será gerado um arquivo CSV que vai atualizando em tempo real, mostrando cada tentativa que foi feita e só para quando a colisão é encontrada, então é uma ótima forma de garantir que tem colisão, o terminal vai mostrando quantas tentativas estão sendo feitas (a cada mil tentativas ele faz um print) e quando acha a colisão é mostrado o hash que se repetiu e as duas strings diferentes, aliás, sobre a string, ela é composta de números e letras, em um espaço de 1 a 6 caracteres, somado com a aleatoriedade isso nos dá um espaço gigante para trabalhar, assim fica o terminal ao final:

```
Tentativas : 105000
Tentativas : 106000
Tentativas : 107000
-----
COLISÃO ENCONTRADA
Hash de Ambos : 9ddf223d
String 1 : 'wRwkvD'
String 2 : 'WzyNw'
```

Então basta seguir até o arquivo CSV e confirmar que é uma colisão usando a ferramenta de busca Ctrl+F, só precisamos colocar o Hash compartilhado e veremos que duas strings no arquivo possuem esse hash, o último a ser escrito e algum outro perdido entre os milhares de registros do arquivo

Essa velocidade de achar dois hashes compartilhados entre strings diferentes prova que reduzir a saída do shake128 para apenas 4 bytes prejudica muito a segurança do algoritmo, validando a necessidade de espaços de saída bem menores para aplicações reais e seguras

Desafio B: Quebrando a Segunda Pré-Imagem

A propriedade da resistência da segunda pré-imagem diz que dado um input x_1 , deve ser computacionalmente difícil achar um segundo input x_2 , tal que $x_1 \neq x_2$ e x_1 e x_2 compartilhem o mesmo hash

Diferentemente do desafio anterior, onde só precisávamos buscar dois hashes iguais, nesse caso temos um valor fixo representado pela string "Aluno: Mateus Lima Rodrigues" e devemos extrair o hash dessa string e comparar com as tentativas

Também diferente do desafio anterior, nesse podemos fazer sem usar memória nenhuma, uma vez que a única coisa que precisamos fazer é gerar um hash de uma string e comparar com o hash do nosso alvo, então o nosso arquivo CSV dessa vez vai armazenar apenas alguns checkpoints de evolução da nossa busca, vamos salvar um checkpoint a cada 500000 tentativas, que vai mostrar a hora exata do salvamento, em qual casa estava o contador de tentativas, o status, ultima string testada e o hash dessa string, a última diferença em código para o anterior foi que para esse precisamos de um espaço de entropia muito maior, uma vez que perdemos a vantagem da probabilidade de "aniversário" que usamos no desafio anterior, nesse estamos percorrendo o espaço de 32 bits, mesmo resolvendo na metade disso, 2^{31} ainda é um espaço gigantesco para ser percorrido

```
-----
Tentativas : 1007000000 / Última String : m1Y5UPwt2ZEN / Hash da String : 214cd895
-----
Tentativas : 1007500000 / Última String : a0VcHuK / Hash da String : ce3d0f97
-----
Tentativas : 1008000000 / Última String : MGbCnG4P7H / Hash da String : 33f0d717
-----
Tentativas : 1008500000 / Última String : RwoelN2GxF2 / Hash da String : 0908028c
-----

SEGUNDA PRÉ-IMAGEM ENCONTRADA
Entrada Alvo(x1) : 'Aluno: Mateus Lima Rodrigues'
Outra String(x2): 'McCvPih'
Hash de Ambos : ac334beb
Tempo Para Encontrar : 3204.22 segundos
```

Acima vemos o log final do terminal, rodou por aproximadamente 1 hora (meu notebook só tem 4gb de RAM) e a string x_2 foi encontrada com o mesmo hash da primeira string, para visualizar podemos usar a mesma ferramenta do desafio anterior, basta ir no arquivo csv e visualizar o hash do último registro e comparar com o hash do nosso alvo, adicionei abaixo do cabeçalho para facilitar

Por fim, esse desafio provou que a resistência da segunda pré-imagem é exponencialmente mais forte que a resistência a colisões do exemplo anterior, ao fazer o desafio C depois desse, percebi que em questão de código eles são bem parecidos(spoiler: rodar o C demorou mais de 2 horas), mas a maior diferença é na teoria por trás dos dois, vou tentar explicar melhor na parte dedicada ao desafio C

Desafio C: Quebrando a Pré-Imagem

A resistência da pré-imagem diz que dado um hash qualquer, deve ser computacionalmente inviável recuperar a mensagem original x , no nosso caso a string que originou esse hash tal que $H(x) = h$, nesse cenário, atuamos como atacantes, dado a tabela de hashes possíveis para serem interceptados, escolhi o hash 4: ae27918d (hashlib exige que seja minúsculo)

O código foi basicamente o mesmo do desafio B, mudando apenas alguns detalhes bem específicos, como por exemplo comparar as strings, no caso do desafio B era necessário comparar a string, pois caso alguma sorte absurda acontecesse e a aleatoriedade conseguisse uma string literalmente igual ao nosso input, o nosso sistema seria quebrado pela mesma string do input, e não por outra diferente, no caso do desafio C não existe essa comparação de strings, pois o que buscamos é exatamente isso, dado o nosso hash, vamos procurar a string que pode ter originado, no caso ou a senha original ou uma colisão, conceito do desafio A

Comparado com o desafio anterior, ambos têm o mesmo grau de dificuldade, nosso alvo é encontrado na maioria das vezes na casa dos bilhões de tentativas, mas os conceitos são bem diferentes, enquanto no desafio B, a segunda pré imagem, queremos algo como uma falsificação, já conhecemos a mensagem original, sendo ela Aluno: Mateus Lima Rodrigues, e dessa mensagem original queremos encontrar outra mensagem que se passe exatamente por ela em conceito de hash, mesmo que a string seja totalmente diferente, enquanto o conceito do desafio C está mais em recuperação, recebemos, ou nesse caso, interceptamos um hash, e desse hash que temos precisamos comparar com várias outras strings que produzem diferentes hashes, até encontrar uma string específica que vai ter o mesmo hash que o que nós recebemos, é como se fosse um código de uso único que várias plataformas estão usando recentemente, você recebe vários códigos para usar se esquecer a senha, todos funcionam como uma senha que você só pode usar uma única vez, nosso exemplo é parecido, é como se pegássemos um banco de dados com várias senhas, mas queremos entrar em um login específico, com esse hash que temos precisamos testar todas as senhas(strings) desse banco até achar uma que seja equivalente à nossa, desse desafio C só não entendi muito bem a descrição de “34 bits”, fiz mantendo o padrão de 32 bits, e realmente não sei como faria funcionar com os 34

Resultado do terminal da última vez que rodei o código, sequestrou a minha memória RAM por 5 horas e meia, e gerou a string 'c2DWbO3T6X', testei no gerador de hash passando pelo shake128 do [codebeautify.org](https://codebeautify.org/shake128-hash-generator) e bateu com a string alvo

```
-----  
Tentativas : 4425000000 / Última String : v / Hash da String : 904f2f17  
-----
```

```
Tentativas : 4425500000 / Última String : a1q4k / Hash da String : 1b6a88c6  
-----
```

```
Tentativas : 4426000000 / Última String : Jj / Hash da String : 38fe8501  
-----  
-----
```

```
PRÉ-IMAGEM ENCONTRADA  
Hash Alvo : AE27918D  
Hash da Última String : ae27918d  
Senha Descoberta : 'c2DWbO3T6X'  
Tempo Decorrido : 20212.06 segundos
```

Conclusão:

Esses 3 desafios sobre o algoritmo shake128 com uma saída truncada para 4 bytes, 32 bits revela falhas críticas de segurança, mesmo que o algoritmo seja muito forte internamente, a segurança desse hash depende muito do tamanho da sua saída

Para explicar de uma forma mais lúdica, podemos voltar e resumir os 3 desafios usando fazer um paralelo com uma investigação criminal:

- Desafio A: Resistência a colisões
 - No nosso paralelo de investigação criminal, buscamos simplesmente duas pessoas que têm a mesma impressão digital, independente de quem seja e independente de qual seja essa impressão digital, basta que encontremos duas pessoas que compartilham a mesma impressão ou duas strings diferentes 1 de mesmo hash
 - Como só buscamos uma conexão, e não um valor específico, o script encontrou rapidamente uma colisão em poucas tentativas, nesse caso o paralelo do aniversário salvou a velocidade
- Desafio B: Resistência a Segunda Pré-Imagem
 - Nesse outro cenário, temos uma string pronta, um nome de alguém específico, nesse caso o objetivo é encontrar outra “pessoa” que tenha a mesma impressão digital que o nosso alvo, assim podemos fazer uma falsificação, tendo a mesma impressão digital do nosso alvo em outra pessoa totalmente diferente
 - Nesse desafio a dificuldade aumentou exponencialmente, uma vez que perdemos a oportunidade de “escolher” um hash para conectar com outro igual, agora temos um hash gerado de uma string e devemos achar outra string que gere o mesmo hash até encontrar o seu sócio em questão de hash
- Desafio C: Resistência da Pré-Imagem
 - Agora, no nosso exemplo final, imagine que a polícia achou uma impressão digital na cena do crime, e extraímos essa impressão, no caso o hash ae27918d, ele é a nossa impressão digital que vamos procurar, então com ele em mãos, vamos testar suspeito por suspeito, testando basicamente o mundo inteiro de 32 bits atrás de alguém que tenha a mesma impressão digital
 - Nos testes, encontrei a string ‘HKyGa3o’ em um teste de aproximadamente 2 horas e a string ‘c2DWbO3T6X’ no último exemplo que fiz, demorou mais de 5 horas, ambas produziram o mesmo hash geradas por um shake128 de 32 bits que a nosso alvo

Finalmente, esses desafios serviram para mostrar que mesmo um dos algoritmos mais estudados e usados pela criptografia não é o mesmo quando submetemos testes em um cenário de pouquíssimos bits, acabando totalmente com a sua segurança, então, como o objetivo era verificar a segurança dessas 3 implementações, podemos agora definir com certeza que elas são completamente inseguras, pois permitem tanto a confusão de dados quanto a falsificação de identidade em tempo hábil

Por fim, os desafios foram estranhamente divertidos e ótimos para a parte prática da cadeira, a parte que mais demorou foi realmente a de escolher como guardar os dados, decidi colocar os conhecimentos da cadeira de persistência em jogo e deixar o csv responsável pelo salvamento do progresso